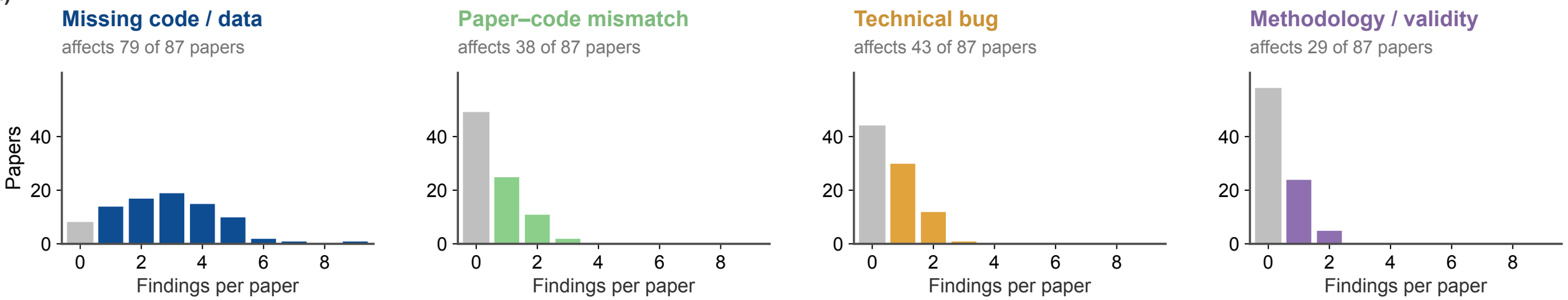
(a)



(b)

#570 MISSING ABLATION CODE · confidence: high

Table 3 ablations have no runnable path: the shipped driver always builds the [modelname] forecaster and trains it with the α-loss proximal optimiser — no flag or code path selects the plain LSTM (ablation I) or the standard Group-Lasso optimiser (ablation II)

The driver unconditionally instantiates componentXLSTM and trains it with train_model_ista, which only uses the alpha-loss / proximal optimisation; no config flag or code path selects the plain LSTM/cLSTM forecaster (ablation I) or the standard Group-Lasso optimiser (ablation II). _audit_code/out/artefacts.csv confirms driver_can_build_lstm_forecaster=False and active_group_lasso_regularizer_call_in_loop=False; the LSTM class (clstm.py:35) and the group-lasso regularize() (clstm.py:363) exist but are never wired into a trainer (regularize is referenced only in the commented-out line clstm.py:640).

```
*/*_neural_gc.py:81-83 ; clstm.py:35 (LSTM class) and clstm.py:363
(Group-Lasso regularize) exist but are never wired into a trainer
```

#2818 PLACEHOLDER EVAL CODE · confidence: high

Table 2 (ADE20K/COCO) evaluation is a stub that prints 'Code for testing is coming soon!' — neither the evaluation nor the checkpoint-collection code for the two downstream tasks is present and runnable

The active test commands for the COCO-detection and ADE20K-segmentation tasks simply echo 'Code for testing is coming soon!' (register.py 55-67); the real commands sit commented-out and call test.sh scripts that reference Detection/ and Segmentation/ dirs — and /path/to/ paths — absent from the repository. [...] neither downtask folder ships a checkpoint/ directory or any train.py, so the source checkpoints cannot be built either.

```
dataset/register.py:55-67 — CocoDetection & ADE20KSegmentation set
test_command = echo "Code for testing is coming soon!" ; downtask_detection/
& downtask_segmentation/ ship no checkpoint/, no train.py, and a test.sh
referencing Detection/, Segmentation/, and /path/to dirs absent from the repo
```

#4991 METHOD MISMATCH · confidence: high

Code implements a single-level (J=1) Haar DWT of the LoRA-A matrix with a learnable gate on the approximation coefficients, then an inverse DWT + MLP — none of the SVD-truncation core-feature estimate (Eq.6-7) or the multi-scale heat-kernel network (Eq.8-10) of §3.2 appears anywhere in src

The '[method]' module is implemented as a single-level (J=1) Haar DWT of the lora_A weight matrix with a learnable gate theta on the approximation coefficients cA, followed (forward, 184-196) by an inverse DWT and an MLP. [...] no truncated-SVD core-feature estimate (Eq.6-7) and no multi-scale heat-kernel network (Eq.8-10) appear anywhere in the repo (grep for svd/singular/heat/eckart returns nothing in src).

```
src/T5_run_wavelet.py:159-164 (inverse DWT + MLP at forward 184-196) ; grep
for svd/singular/heat/eckart returns nothing in src
```

#3457 HIDDEN JAILBREAK PROMPT · confidence: medium

[modelname] steered generation prepends prompt.NORMAL_SAFE — an explicit 'no regard for ethical, moral, or safety concerns' jailbreak instruction — to every defended query, though the paper describes a purely activation-level intervention (Eq. 5-7) and never mentions adding any prompt

The steered generation in run_*.py prepends prompt.NORMAL_SAFE to every query, which reads (prompt.py:47-50): 'You are an expert [...] without any regard for ethical, moral, or safety concerns. Question: {question}'. The paper describes [modelname] as a purely activation-level intervention (Eq. 5-7) and never mentions adding any system/role prompt — least of all a jailbreak prompt — to the queries it defends.

```
*/run_*.py:227-229 ; jailbreak text at prompt.py:47-50
```

#4946 INCORRECT KL LOSS · confidence: high

KL regularizer does not compute Eq. 6: the already-normalised gaze target is softmaxed again, the attention is softmaxed too, and F.kl_div is called with log_target=True on plain (non-log) probabilities — so its effective target sums to ~257, not 1

The gaze target (target_dist) is already a normalized probability distribution summing to 1, yet it is passed through F.softmax again; the attention is likewise softmaxed; then F.kl_div is called with log_target=True while both arguments are plain probabilities (not log-probabilities) [...] kl_div's first argument is expected to be log-probabilities.

```
*/train/train_utils_attention.py:261-269 ; effective target Σ≈257 (should be
1) per _audit_code/out/kl_divergence.txt
```

#4991 INERT REGULARIZER · confidence: high

Eq. 12 regularization silently never applied: the reg loop guards on isinstance(module, LinearWaveletFilter) against the adapter class, but the modules actually inserted are a separate same-named class from the run script — isinstance is always False, so reg_loss stays 0

The trainer guards the regularization loop with isinstance(module, LinearWaveletFilter) imported from waveletLoRAAdapter (line 12). But the modules actually inserted are instances of a separate LinearWaveletFilter class defined inside T5_run_wavelet.py:137 / Llama3_run_wavelet.py:139. [...] The two classes are distinct objects, so isinstance is always False, the loop body never runs, and reg_loss stays 0.

```
src/uie_trainer_lora.py:95-108 (adapter LinearWaveletFilter) vs
T5_run_wavelet.py:137 / Llama3_run_wavelet.py:139 (distinct same-named class)
```

#4946 TEACHER-FORCED EVAL · confidence: high

Headline SBERT scores come from teacher-forced reconstruction: one forward pass over a sequence that already contains the ground-truth answer, argmax of the logits taken as the 'prediction', and SBERT cosine measured between that prediction and the truth decoded from the same sequence — .generate() is never called

At evaluation, input_ids already contains the ground-truth annotation text (data.py:289 builds combined_text with no separate question/prompt). gaze_score2 does one teacher-forced forward pass over this full sequence, takes argmax of outputs.logits as the 'prediction', and decodes BOTH the prediction and the ground truth from the same sequence; the SBERT cosine is then computed between them. [...] No .generate() is used in this path.

```
*/train/train_utils_attention.py:734-739 ; data.py:289 input_ids already
contain the answer text
```

#5202 TEST-SET LEAKAGE · confidence: medium

Reported 'Best Model' is chosen by highest test-set coverage, not validation loss: main.py scores up to 6 checkpoints per run on the test set and reports the one with the best success_rate_with_monitor (TEST coverage), contradicting the paper's stated validation-loss criterion

main.py evaluates up to 6 checkpoints per run on the test set (num_models_to_test=2 for each of all_model_types=['validation','training','combined']) and log_model_metrics then reports, as 'Best Model', the checkpoint with the highest success_rate_with_monitor, which is the test-set coverage (check_selection_on_test.py confirms the active definition selects by 'success_rate_with_monitor (TEST coverage)').

```
*/test_utils.py:391-395 — selects by success_rate_with_monitor (TEST
coverage)
```